

Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Teoría

Índice

0 - Introducción a los Microcontroladores	3
0.1 - Integración y SoC	3
0.2 - Clasificación de MCUs	3
0.3 - Conexionado y Funcionamiento Básico	3
0.4 - Consumo y Energía	3
0.5 - Ecosistema de Desarrollo	3
0.6 - Introducción a Kinetis K64F	3
0.6.1 - La Familia Kinetis	3
0.6.2 - Microcontrolador K64	3
0.6.3 - Placa de Evaluación: FRDM-K64F	3
0.6.4 - Toolchain: MCUXpresso	4
1 - Clase 1	5
1.1 - Buses de datos y de direcciones	5
1.2 - GPIO: entradas y salidas digitales	6
1.2.1 - Readback: leer el estado de una salida	7
1.2.2 - Active-low y active-high	7
1.2.3 - Propiedades eléctricas de un pin	8
1.2.4 - Configuraciones de pines con MOSFETS	9
1.3 - GPIO del K64	9
1.3.1 - PDDR	11
1.3.2 - PSOR	11
1.3.3 - PCOR	11
1.3.4 - PTOR	12
1.3.5 - PDIR	12
1.4 - PCR: configuración individual de cada pin	12
1.4.1 - PCR - MUX	14
1.4.2 - Secuencia para configurar un pin como GPIO	14
1.5 - PCR - Global	15
1.5.1 - Interrupt Status Flag Register (PORTx_ISFR)	15
1.6 - Digital Filter	16
1.7 - Clock Gating	16
1.8 - Proramming PORTx_ISFR	17
1.9 - PORT_type	18
2 - Clase 2	19
2.1 - Bit Band	19
2.2 - Interrupciones	20
2.2.1 - Máscaras	20
2.2.2 - Interrupt Vectors	20
2.2.3 - Flags	20
2.2.4 - Secuencia de la Interrupcion	21
2.2.5 - Código Bloqueante vs Pooling	21
2.2.6 - Periodic Interrupt / Hardware Polling	22
2.2.7 - Interrupciones Dedicadas	22
2.2.8 - Fuentes de interrupción periódica	23
2.3 - Interrupciones del K64	24
2.3.1 - NVIC / Nested Vector Interrupt Controller .	24
2.3.2 - Weak	25
2.3.3 - CMSIS - Functions	25
2.3.4 - Ejemplo de Implementacion de Interrupcion .	27
3 - Clase 3	30
3.1 - Prioridad de Interrupciones	30
4 - Clase 4	32
4.1 - Arquitectura de Firmware	32
4.1.1 - Capas e Interfaces	32
4.2 - Estructura de Código	32
4.2.1 - DRV_Init()	32
4.2.2 - DRV_SrvN()	33
4.2.3 - DRV_PISR()	33
4.2.4 - DRV_ISR()	33
4.3 - Analisis temporal	33
4.3.1 - Código Bloqueante	33
4.3.2 - Polling	33
4.3.3 - PISR	33
4.3.4 - ISR (Interrupcion dedicada)	34
4.3.5 - ISR + callback	34

0 - Introducción a los Microcontroladores

0.1 - Integración y SoC

Históricamente, los sistemas microprocesados requerían múltiples integrados (CPU, RAM, ROM, Clock, etc.) . Los **Microcontroladores (MCU)** actuales integran todos estos bloques en un solo chip, conocido como **System on Chip (SoC)** :

- **CPU:** Microprocesador central.
- **Memoria:** RAM para datos y Flash/ROM para programa.
- **Periféricos:** Módulos de hardware (Timers, ADC, UART) configurables mediante registros que funcionan en paralelo a la CPU, permitiendo multitarea real .

0.2 - Clasificación de MCUs

Se eligen y distinguen principalmente por :

- **CPU:** Arquitecturas de 8, 16 o 32 bits.
- **Memoria:** Cantidad de RAM y FLASH disponible.
- **Encapsulado:** Número y tipo de pines físicos.
- **Aplicación:** Bajo costo, alta performance o bajo consumo (**Low Power**).

0.3 - Conexión y Funcionamiento Básico

- **Alimentación:** Pines VDD y GND. La tendencia actual es bajar la tensión (3.3V, 1.8V) para reducir el consumo .
- **Clock:** Provee la señal de sincronismo. Muchos poseen PLL internos para multiplicar la frecuencia del oscilador .
- **Reset:** Asegura que el programa inicie desde una condición predefinida (pin nRESET) .
- **Entradas/Salidas (I/O):**
- **GPIO:** Pines digitales configurables (entrada/salida, pull-up/down, etc.) .
- **Analógicos:** Conversores A/D y D/A para el mundo real .

0.4 - Consumo y Energía

El consumo se divide en potencia estática y dinámica, dominando generalmente la **dinámica** . Es fundamental el uso de modos **Sleep** para ahorrar energía en sistemas a batería, teniendo en cuenta que a menor consumo, mayor es el tiempo de despertar (**wakeup**).

0.5 - Ecosistema de Desarrollo

Para trabajar con un MCU se requiere:

1. **Documentación:** Datasheet y Reference Manual.
2. **Evaluation Board:** Placa de desarrollo con el MCU.
3. **Toolchain:** IDE, compilador y programador/debugger.
4. **SDK:** Librerías y ejemplos de código.

0.6 - Introducción a Kinetis K64F

0.6.1 - La Familia Kinetis

Kinetis es una familia de microcontroladores de **32 bits de NXP** basada en el núcleo **ARM Cortex-M** .

0.6.2 - Microcontrolador K64

El modelo específico de estudio es el **MK64FN1M0VLL12** . Sus características principales son :

- **Core:** ARM Cortex-M4 a 120 MHz con FPU (Unidad de Punto Flotante).
- **Memoria:** 1 MB de Flash y 256 KB de SRAM.
- **Encapsulado:** 100 LQFP.

0.6.3 - Placa de Evaluación: FRDM-K64F

Es una plataforma de bajo costo que incluye :

- **Conectividad:** Ethernet, USB (Host/Device) y slot para MicroSD (SDHC).
- **Sensores:** Acelerómetro y magnetómetro integrados.
- **Interfaz Humana:** LED RGB y 2 pulsadores (SW2 y SW3).
- **Depuración:** Programador incorporado **OpenSDAv2**.

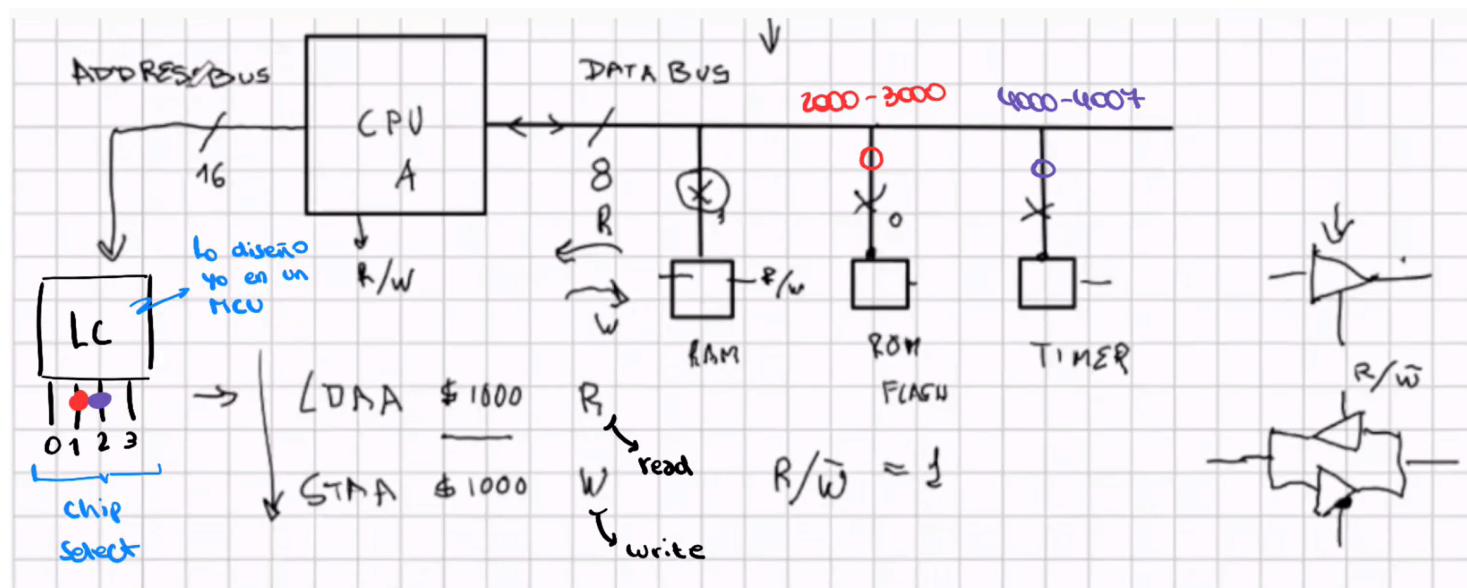
0.6.4 - Toolchain: MCUXpresso

El IDE oficial de NXP para esta plataforma es **MCUXpresso**, basado en Eclipse . Para su funcionamiento con la familia K64, es indispensable instalar el **SDK específico** de la placa FRDM-K64F .

Nota sobre Registros: El control de los periféricos en el K64 se realiza mediante la escritura en registros de 32 bits, como el **Pin Control Register** (PCR), que define la multiplexación (MUX) y características eléctricas de cada pin .

1 - Clase 1

1.1 - Buses de datos y de direcciones



Con una instrucción como LDAA \$1000, la CPU solicita el contenido de la dirección de memoria \$1000 y lo carga en el acumulador A. La dirección viaja por el **Address Bus** y el dato leído vuelve por el **Data Bus**.

El **Address Bus** indica **con qué dispositivo** se quiere comunicar la CPU y el **Data Bus** transporta **qué valor** se lee o se escribe. Ambos buses trabajan coordinadamente durante cada acceso a memoria.

Para evitar que varios periféricos intenten conducir el **Data Bus** al mismo tiempo, sus salidas utilizan buffers **tri-state**. La lógica de control habilita un único dispositivo durante la transferencia:

- R/W indica si la operación es de lectura (**Read**) o escritura (**Write**).
- CS (**Chip Select**) habilita el periférico seleccionado.
- Un dispositivo no seleccionado desconecta eléctricamente su salida del bus y queda en **alta impedancia (Z)**.

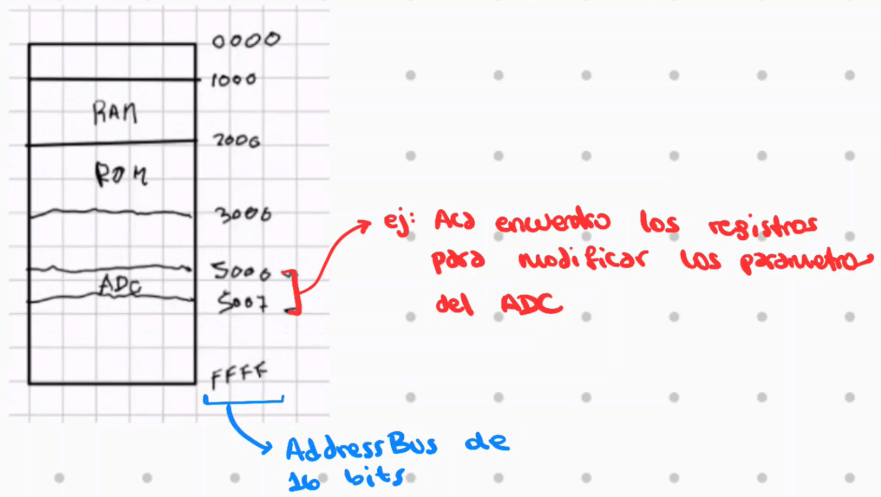
Baja impedancia: el circuito está conectado al bus y puede imponer un nivel lógico.

Alta impedancia (Z): el circuito se comporta como si estuviera desconectado del bus.

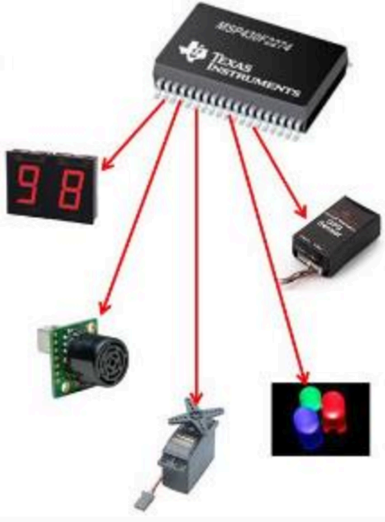
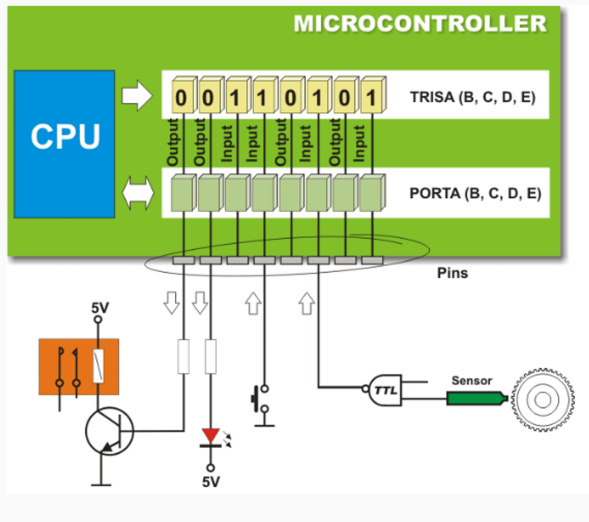
El Address Bus llega a la **lógica de decodificación (LC)**, una lógica combinacional que observa la dirección y genera señales de selección. Según el rango de direcciones, activa el CS del periférico correspondiente.

Por ejemplo, si las direcciones \$1000 a \$2000 están asignadas a la RAM, una dirección dentro de ese rango activa CS_RAM. La CPU no “habla” directamente con todos los periféricos: la decodificación determina cuál puede responder.

La asignación de rangos y la lógica de decodificación forman parte del diseño del MCU.



1.2 - GPIO: entradas y salidas digitales



Los pines de un GPIO se controlan mediante registros mapeados en memoria. Cada registro tiene una dirección propia, por lo que la CPU puede acceder a él usando el mismo mecanismo de buses explicado anteriormente.

En este ejemplo, P1DIR determina la dirección de cada pin: un bit en 0 configura el pin como entrada y un bit en 1 lo configura como salida. El circuito se repite para cada bit del puerto.

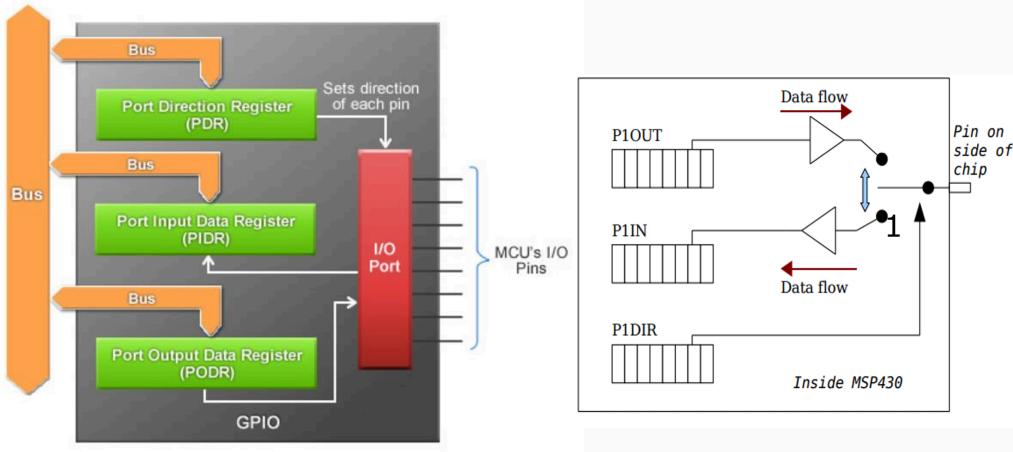


Figura 5: Ejemplo Generico

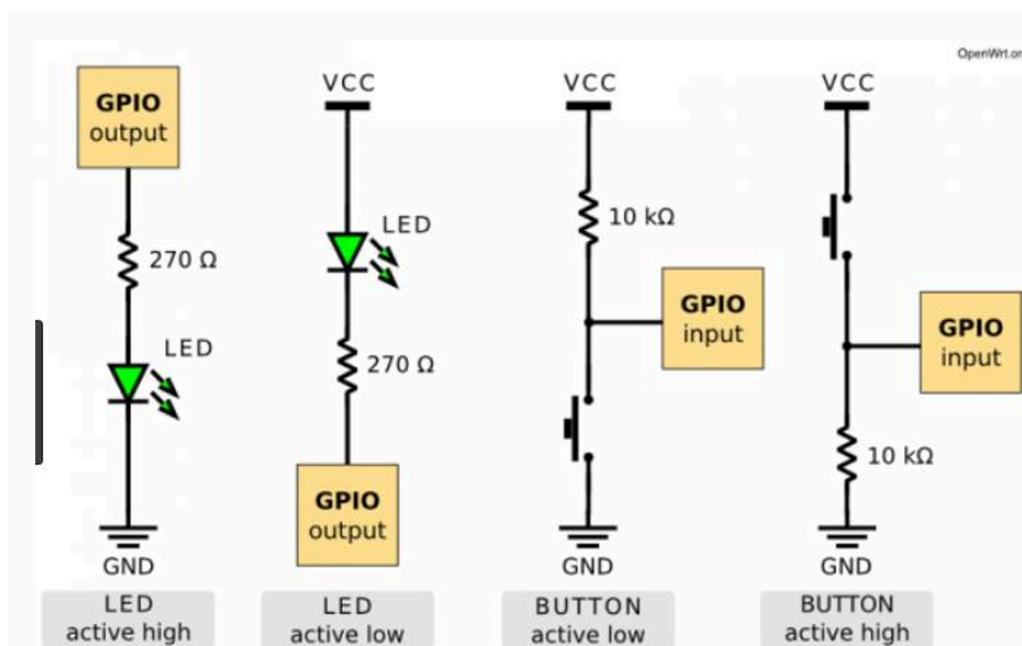
1.2.1 - Readback: leer el estado de una salida

El **readback** es una técnica para verificar el estado de una señal usando un pin de lectura. Se configura un pin como OUTPUT, que genera la señal, y otro como INPUT, que la observa.

Esto resulta útil para detectar si una salida realmente cambia de estado o si existe una falla en el circuito externo. Los dos pines deben estar conectados correctamente y nunca se deben configurar dos salidas con valores opuestos sobre el mismo conductor.



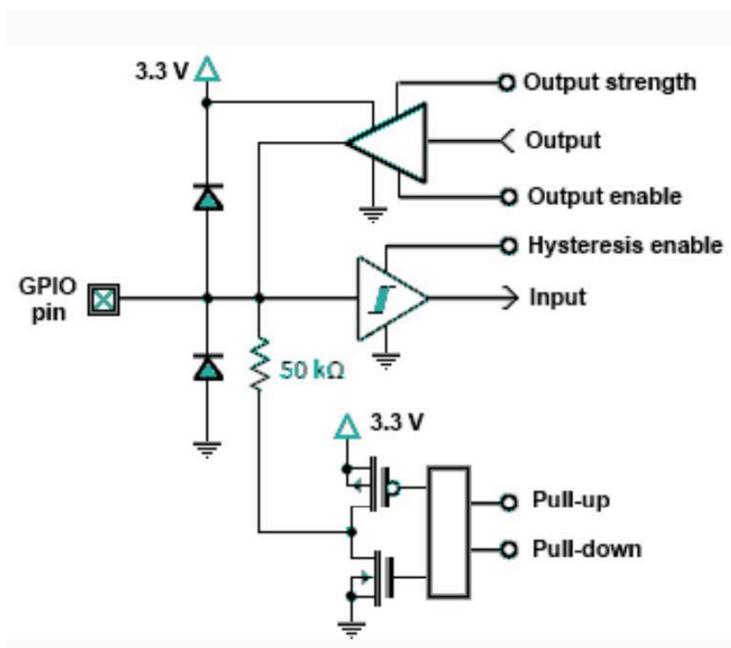
1.2.2 - Active-low y active-high



Una señal es **active-high** cuando se considera activa en nivel lógico 1. Es **active-low** cuando se considera activa en nivel lógico 0; suele indicarse con una barra superior, un sufijo `_n` o una burbuja en el esquema.

Por ejemplo, un `LED_EN_n` habilita el LED cuando vale 0. El nivel lógico y el significado funcional no siempre coinciden: 0 puede significar “activo”.

1.2.3 - Propiedades eléctricas de un pin

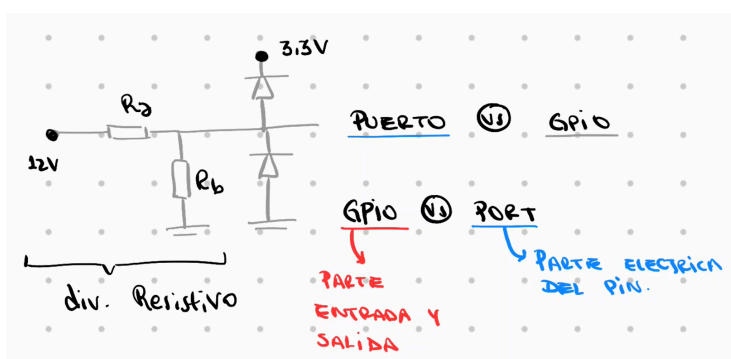


Los pines suelen incluir diodos de protección. Estos comienzan a conducir cuando la tensión supera aproximadamente V_{DD} o cae por debajo de V_{SS} (según el circuito del microcontrolador). No deben utilizarse como fuente normal de protección: hay que respetar los límites eléctricos del **datasheet**.

También se puede configurar una resistencia interna PULL-UP o PULL-DOWN. Su función es evitar que una entrada quede **flotante** cuando ningún circuito externo fija su nivel lógico.

Es posible configurar por software los pines como INPUT o OUTPUT, pero esa configuración debe coincidir con el circuito conectado. Para observar la diferencia con un osciloscopio, una salida presenta baja impedancia y domina el nivel del pin. Una entrada presenta alta impedancia y puede captar ruido; al acercar un dedo puede observarse una señal de red de aproximadamente 50 Hz.

Si una señal externa está fuera del rango permitido del pin, no alcanza con configurarla como entrada. Es necesario adaptar la tensión, por ejemplo con un divisor resistivo y, cuando corresponda, diodos o un circuito de protección dimensionado para la aplicación.



Drive strength (DSE): es la capacidad de corriente de la salida. Define qué carga puede manejar el pin a una velocidad determinada sin degradar la integridad de la señal.

Una mayor capacidad de manejo no siempre es mejor: puede aumentar el consumo y el ruido electromagnético. Se debe elegir la configuración mínima que cumpla con los tiempos y la carga requeridos.

1.2.4 - Configuraciones de pines con MOSFETS

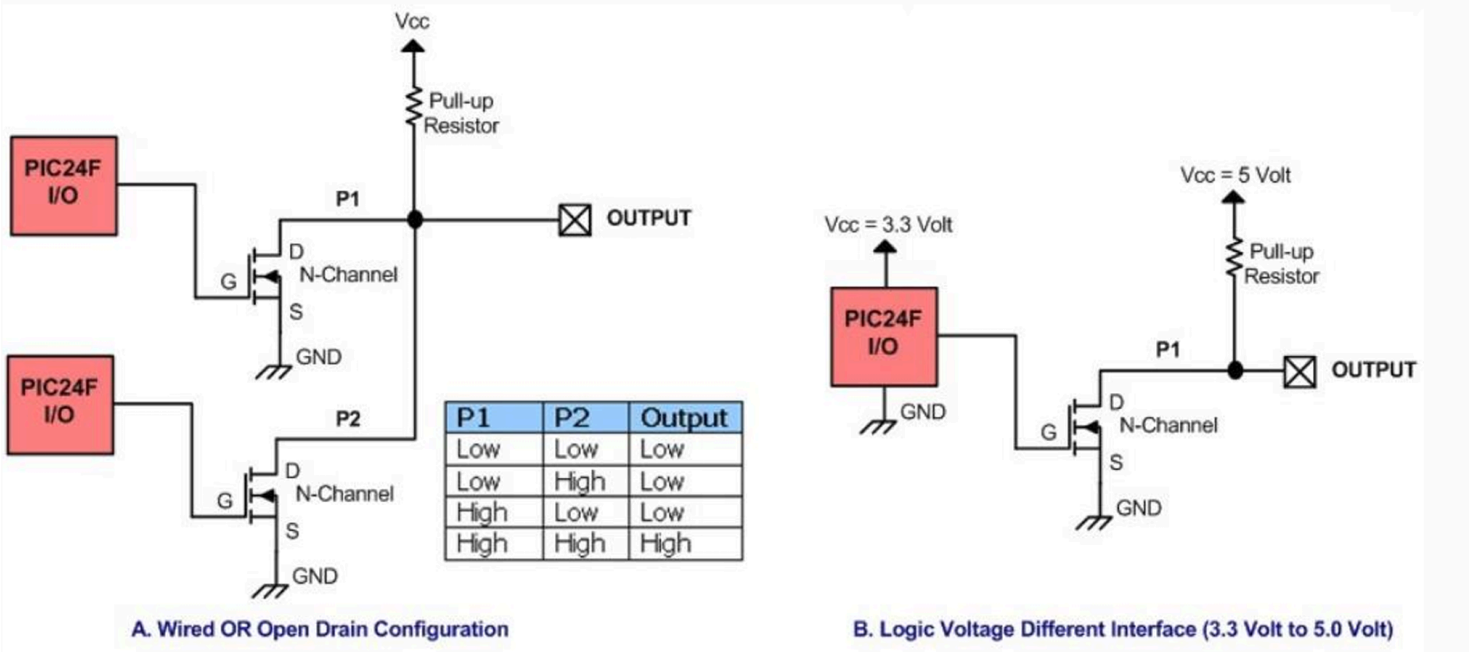


Figura 10: Implementación de lógica digital con MOSFET y elevación de la tensión de salida

1.3 - GPIO del K64

El K64 contiene cinco puertos GPIO de 32 bits cada uno. No todos los pines necesariamente están disponibles en el encapsulado o cumplen la función GPIO, por lo que siempre hay que consultar el **datasheet**.

- PORTA [31:0]
- PORTB [31:0]
- PORTC [31:0]
- PORTD [31:0]
- PORTE [31:0]

PORT memory map

Absolute address (hex)	Register name	Width (in bits)
4004_9000	Pin Control Register n (PORTA_PCR0)	32
4004_9004	Pin Control Register n (PORTA_PCR1)	32
4004_9008	Pin Control Register n (PORTA_PCR2)	32
4004_900C	Pin Control Register n (PORTA_PCR3)	32
4004_9010	Pin Control Register n (PORTA_PCR4)	32
4004_9014	Pin Control Register n (PORTA_PCR5)	32
4004_9018	Pin Control Register n (PORTA_PCR6)	32
4004_901C	Pin Control Register n (PORTA_PCR7)	32
4004_9020	Pin Control Register n (PORTA_PCR8)	32
4004_9024	Pin Control Register n (PORTA_PCR9)	32
4004_9028	Pin Control Register n (PORTA_PCR10)	32
4004_902C	Pin Control Register n (PORTA_PCR11)	32
4004_9030	Pin Control Register n (PORTA_PCR12)	32
4004_9034	Pin Control Register n (PORTA_PCR13)	32
4004_9038	Pin Control Register n (PORTA_PCR14)	32
4004_903C	Pin Control Register n (PORTA_PCR15)	32
4004_9040	Pin Control Register n (PORTA_PCR16)	32
4004_9044	Pin Control Register n (PORTA_PCR17)	32
4004_9048	Pin Control Register n (PORTA_PCR18)	32
4004_904C	Pin Control Register n (PORTA_PCR19)	32
4004_9050	Pin Control Register n (PORTA_PCR20)	32
4004_9054	Pin Control Register n (PORTA_PCR21)	32
4004_9058	Pin Control Register n (PORTA_PCR22)	32
4004_905C	Pin Control Register n (PORTA_PCR23)	32

4004_9060	Pin Control Register n (PORTA_PCR24)	32
4004_9064	Pin Control Register n (PORTA_PCR25)	32
4004_9068	Pin Control Register n (PORTA_PCR26)	32
4004_906C	Pin Control Register n (PORTA_PCR27)	32
4004_9070	Pin Control Register n (PORTA_PCR28)	32
4004_9074	Pin Control Register n (PORTA_PCR29)	32
4004_9078	Pin Control Register n (PORTA_PCR30)	32
4004_907C	Pin Control Register n (PORTA_PCR31)	32
4004_9080	Global Pin Control Low Register (PORTA_GPCLR)	32
4004_9084	Global Pin Control High Register (PORTA_GPCHR)	32
4004_90A0	Interrupt Status Flag Register (PORTA_ISFR)	32
4004_90C0	Digital Filter Enable Register (PORTA_DFER)	32
4004_90C4	Digital Filter Clock Register (PORTA_DFCR)	32
4004_90C8	Digital Filter Width Register (PORTA_DFWR)	32

Esto se repite para los puertos B,C,D,E

Figura 11: Kinetis K64 - Reference Manual - Chapter 4

Cada puerto dispone de registros de datos de 32 bits. La X representa el puerto elegido (A, B, C, D o E):

- GPIOX_PDR (**Port Data Output Register**): valor que se envía a los pines configurados como salida.
- GPIOX_PSOR (**Port Set Output Register**): escribe 1 en los bits indicados del PDR.
- GPIOX_PCOR (**Port Clear Output Register**): escribe 0 en los bits indicados del PDR.
- GPIOX_PTOR (**Port Toggle Output Register**): invierte los bits indicados del PDR.
- GPIOX_PDIR (**Port Data Input Register**): permite leer el nivel observado en los pines.
- GPIOX_PDDR (**Port Data Direction Register**): 0 configura entrada y 1 configura salida.

Los registros PSOR, PCOR y PTOR permiten modificar bits sin hacer una operación de lectura-modificación-escritura sobre PDR. Esto es especialmente útil cuando una interrupción u otra parte del programa también puede acceder al puerto.

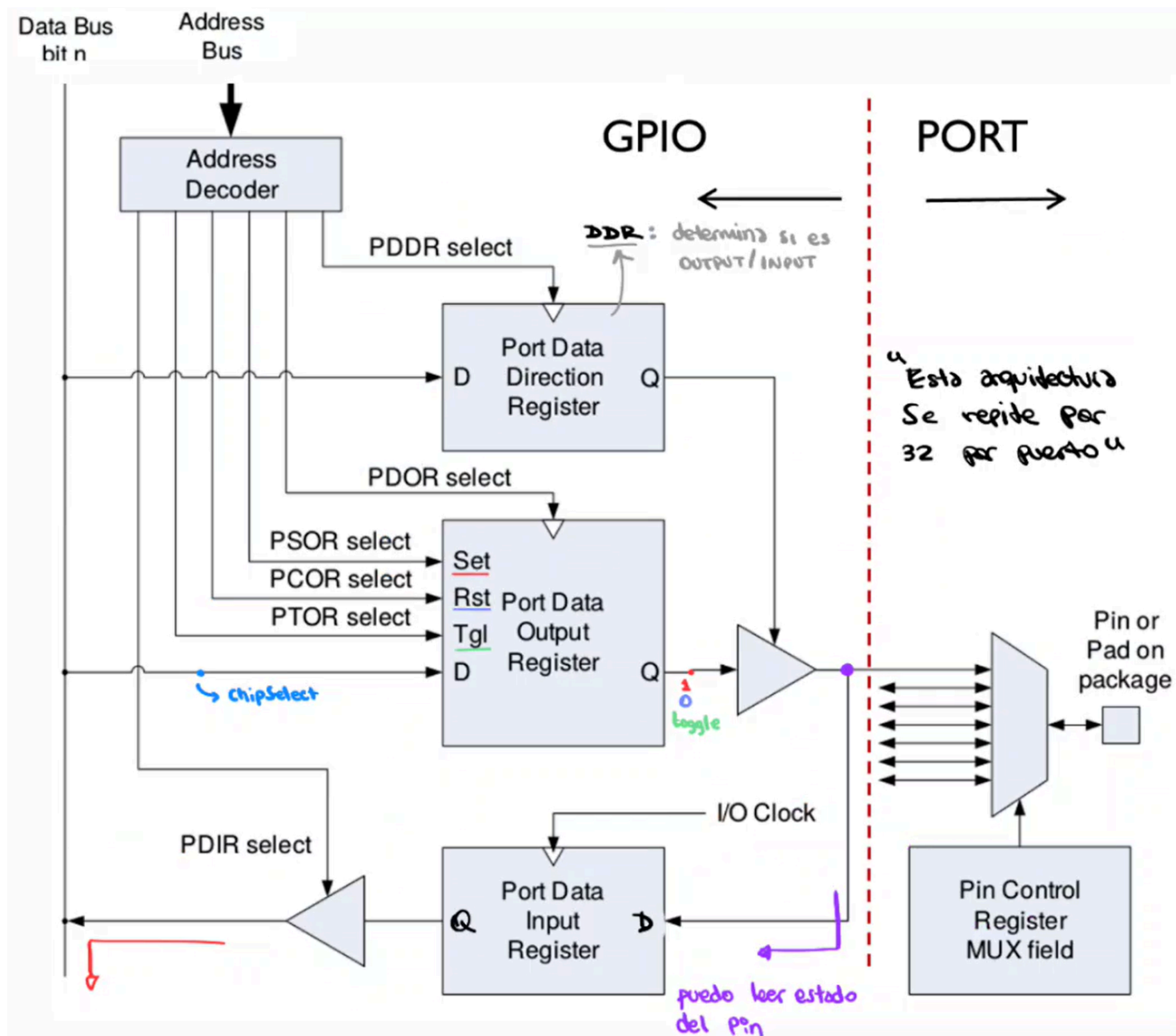


Figura 12: Registros de datos de un puerto GPIO

1.3.1 - PDDR

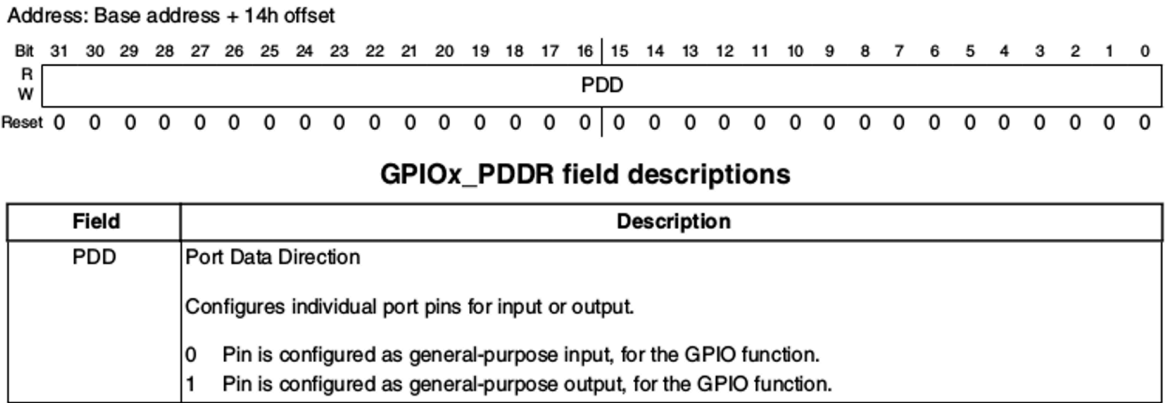


Figura 13: Configuración de dirección mediante PDDR

1.3.2 - PSOR

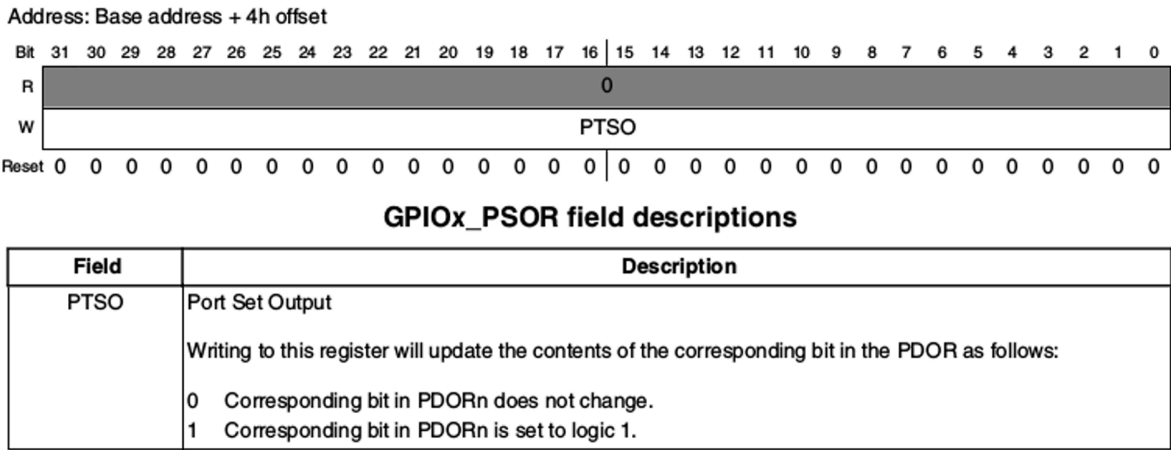


Figura 14: Puesta en uno de bits mediante PSOR

1.3.3 - PCOR

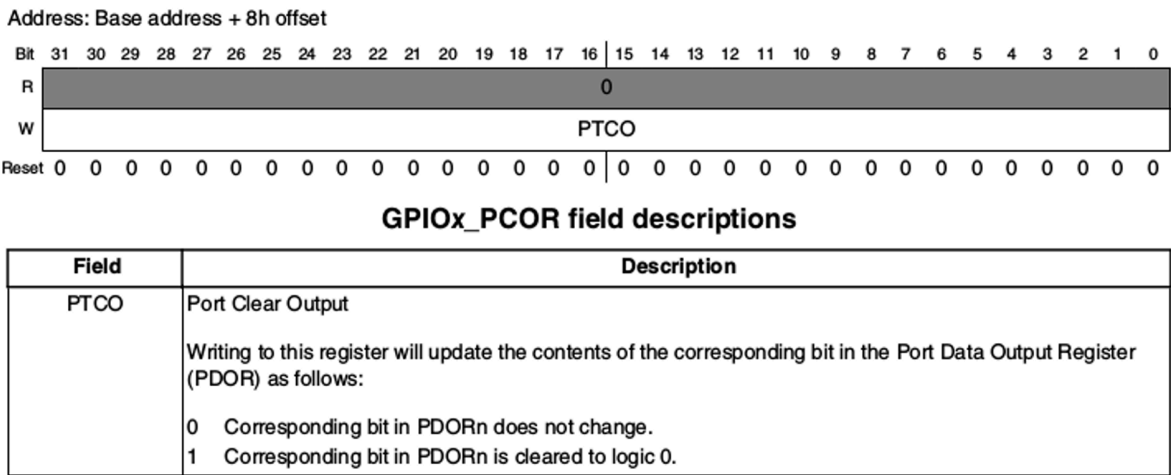
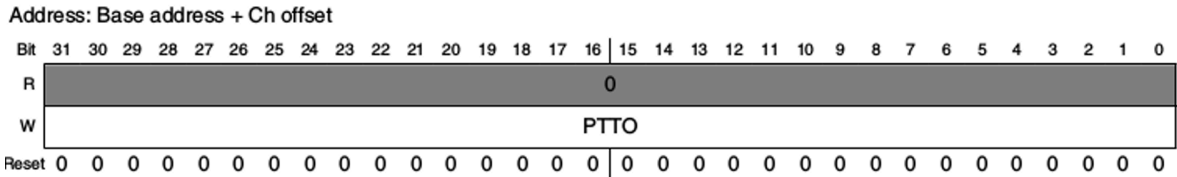


Figura 15: Puesta en cero de bits mediante PCOR

1.3.4 - PTOR

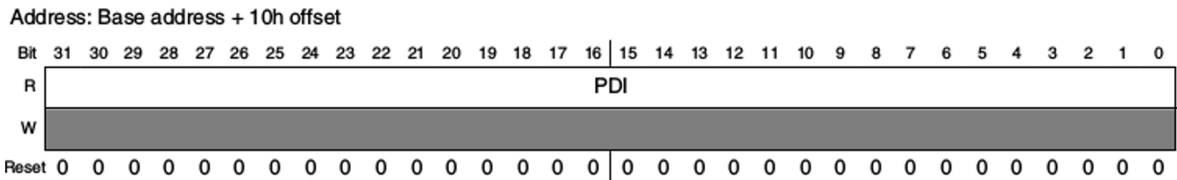


GPIOx_PTOR field descriptions

Field	Description
PTTO	Port Toggle Output Writing to this register will update the contents of the corresponding bit in the PDOR as follows: 0 Corresponding bit in PDORn does not change. 1 Corresponding bit in PDORn is set to the inverse of its existing logic state.

Figura 16: Conmutación de bits mediante PTOR

1.3.5 - PDIR



GPIOx_PDIR field descriptions

Field	Description
PDI	Port Data Input Reads 0 at the unimplemented pins for a particular device. Pins that are not configured for a digital function read 0. If the Port Control and Interrupt module is disabled, then the corresponding bit in PDIR does not update. 0 Pin logic level is logic 0, or is not configured for use by digital function. 1 Pin logic level is logic 1.

Figura 17: Lectura de entradas mediante PDIR

1.4 - PCR: configuración individual de cada pin

Cada pin de cada puerto se configura mediante un registro de 32 bits llamado **Pin Control Register** (PCR). Este registro pertenece a la capa PORT, mientras que los registros PDOR, PDIR y relacionados pertenecen a la capa GPIO.

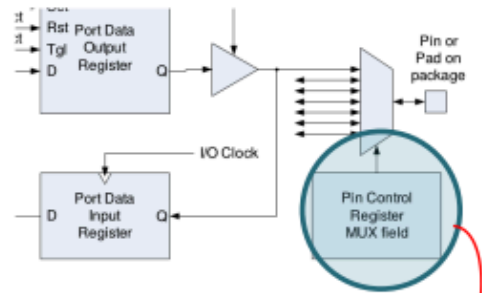
1.4.1 - PCR - MUX

Pin Mux Control

Not all pins support all pin muxing slots. Unimplemented pin muxing slots are reserved and may result in configuring the pin for a different pin muxing slot.

The corresponding pin is configured in the following pin muxing slot as follows:

- 000 Pin disabled (analog).
- 001 Alternative 1 (GPIO).
- 010 Alternative 2 (chip-specific).
- 011 Alternative 3 (chip-specific).
- 100 Alternative 4 (chip-specific).
- 101 Alternative 5 (chip-specific).
- 110 Alternative 6 (chip-specific).
- 111 Alternative 7 (chip-specific).



Address: Base address + 0h offset + (4d × i), where i=0d to 31d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0							ISF	0							IRQC
W								w1c								
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	LK	0				MUX			0	DSE	ODE	PFE	0	SRE	PE	PS
W																
Reset	0	0	0	0	0	*	*	*	0	*	0	*	0	*	*	*

144	144	121	100	Pin Name	Default	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5	ALT6	ALT7	EzPort
LQFP	MAP BGA	XFBG A	LQFP											
62	M9	J9	—	PTA10	DISABLED		PTA10		FTM2_CH0	MIIO_RXD2		FTM2_QD_PHA	TRACE_D0	
63	L9	J4	—	PTA11	DISABLED		PTA11		FTM2_CH1	MIIO_RXCLK	I2C2_SDA	FTM2_QD_PHB		
64	K9	K8	42	PTA12	CMP2_IN0	CMP2_IN0	PTA12	CAN0_TX	FTM1_CH0	RMII0_RXD1/MIIO_RXD1	I2C2_SCL	I2S0_TXD0	FTM1_QD_PHA	
65	J9	L8	43	PTA13/LLWU_P4	CMP2_IN1	CMP2_IN1	PTA13/LLWU_P4	CAN0_RX	FTM1_CH1	RMII0_RXD0/MIIO_RXD0	I2C2_SDA	I2S0_TX_FS	FTM1_QD_PHB	
66	L10	K9	44	PTA14	DISABLED		PTA14	SPI0_PCS0	UART0_TX	RMII0_CRS_DV/MIIO_RXDV	I2C2_SCL	I2S0_RX_BCLK	I2S0_TXD1	

Figura 21: Kinetis K64F Sub-Family Data Sheet - Pin Out

1.4.2 - Secuencia para configurar un pin como GPIO

Una secuencia típica de inicialización es:

1. Habilitar el reloj del puerto en SIM->SCGC5.
2. Configurar PORTX_PCRn con el MUX correspondiente y las características eléctricas necesarias.
3. Definir la dirección en GPIOX_PDDR: 0 para entrada o 1 para salida.

4. Escribir o leer el estado mediante PDOR, PSOR, PCOR, PTOR o PDIR.

El orden es importante: un registro del periférico no puede utilizarse correctamente si su reloj no fue habilitado previamente.

1.5 - PCR - Global

PINS[15 to 0]

PINS[31 to 16]

Address: Base address + 00h offset

Bit 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

R 0

W GPWE GPWD

Reset 0

Mask for PCR[15:0]

Address: Base address + 84h offset

Bit 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

R 0

W GPWE GPWD

Reset 0

Mask for PCR[31:16]

Address: Base address + 0h offset + (4d x i), where i=0d to 31d

Bit 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16

R 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

W 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Reset 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

ISF w1c IRQC

Bit 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

R LK 0 MUX 0 DSE ODE PFE 0 SRE PE PS

W 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Reset 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Data for GPIOx[15:0]

(PORTx_GPCLR)

Low

(PORTx_GPCHR)

High

Permite Modificar en forma simultanea varios PCR del puerto X (n=0:15 LOW) (n=16:31 HIGH)

Global Pin Control Low Register: PORTx_GPCLR

Address: Base address + 80h offset

Bit 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

R 0

W GPWE GPWD

Reset 0

Mask for GPIOx[15:0] Data for GPIOx[15:0]

(PORTx_GPCLR)

PORTx_GPCLR field descriptions

Field	Description
31–16 GPWE	Global Pin Write Enable Selects which Pin Control Registers (15 through 0) bits [15:0] update with the value in GPWD. If a selected Pin Control Register is locked then the write to that register is ignored. 0 Corresponding Pin Control Register is not updated with the value in GPWD. 1 Corresponding Pin Control Register is updated with the value in GPWD.
15–0 GPWD	Global Pin Write Data Write value that is written to all Pin Control Registers bits [15:0] that are selected by GPWE.

Global Pin Control High Register: PORTx_GPCHR

Only 32-bit writes are supported to this register.

Address: Base address + 84h offset

Bit 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

R 0

W GPWE GPWD

Reset 0

Mask for GPIOx [31:16] Data for GPIOx [15:0]

(PORTx_GPCHR)

PORTx_GPCHR field descriptions

Field	Description
31–16 GPWE	Global Pin Write Enable Selects which Pin Control Registers (31 through 16) bits [15:0] update with the value in GPWD. If a selected Pin Control Register is locked then the write to that register is ignored. 0 Corresponding Pin Control Register is not updated with the value in GPWD. 1 Corresponding Pin Control Register is updated with the value in GPWD.
15–0 GPWD	Global Pin Write Data Write value that is written to all Pin Control Registers bits [15:0] that are selected by GPWE.

1.5.1 - Interrupt Status Flag Register (PORTx_ISFR)

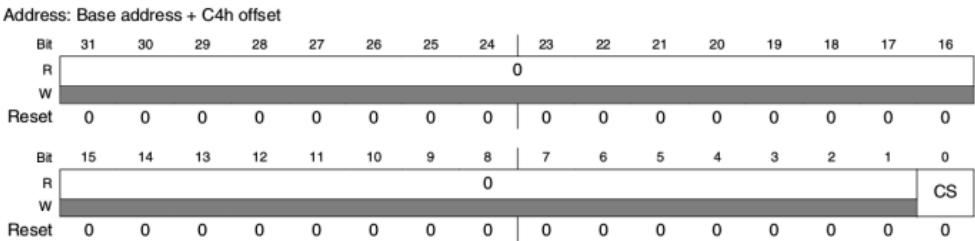
El registro PORTx_ISFR (Interrupt Status Flag Register) es una herramienta de hardware que facilita la identificación de qué pin generó una interrupción dentro de un puerto específico.

15 of 34

1.6 - Digital Filter

El filtrado de las entradas digitales permite rechazar pulsos no deseados presentes en líneas digitales que normalmente están en 0 o 1. Estas interferencias pueden provenir de fuentes de RF y/o conmutadores electromecánicos, etc.

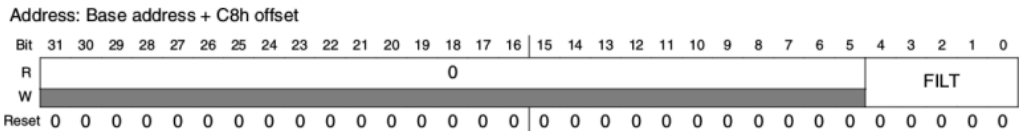
Digital Filter Clock Register (PORTx_DFCR)



PORTx_DFCR field descriptions

Field	Description
31-1 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
0 CS	Clock Source The digital filter configuration is valid in all digital pin muxing modes. Configures the clock source for the digital input filters. Changing the filter clock source must be done only when all digital filters are disabled. 0 Digital filters are clocked by the bus clock. 1 Digital filters are clocked by the 1 kHz LPO clock.

Digital Filter Width Register (PORTx_DFWR)



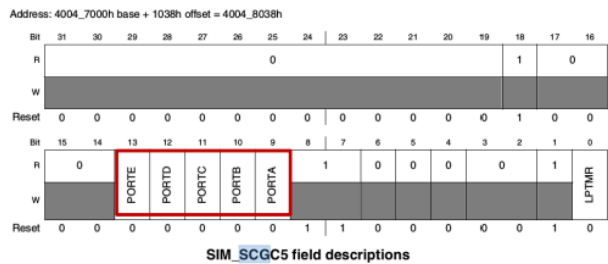
PORTx_DFWR field descriptions

Field	Description
31-5 Reserved	This field is reserved. This read-only field is reserved and always has the value 0.
4-0 FILT	Filter Length The digital filter configuration is valid in all digital pin muxing modes. Configures the maximum size of the glitches, in clock cycles, that the digital filter absorbs for the enabled digital filters. Glitches that are longer than this register setting will pass through the digital filter, and glitches that are equal to or less than this register setting are filtered. Changing the filter length must be done only after all filters are disabled.

1.7 - Clock Gating

- Los procesadores modernos poseen mecanismos para reducir el consumo del chip apagando todos los recursos que no son usados. Esto se logra apagando el clock de dichos módulos (clock gating).
- Para configurar que módulos reciben clock se deben habilitar los bits correspondientes en los registros SCGCx (System Clock Gating Control Register x).
- Después del reset estos bits están apagados para ahorrar energía.
- Los SCGCx. se encuentran dentro del modulo SIM (System Integration Module).
- Antes de usar un modulo se debe habilitar el clock de lo contrario se genera un error. De igual forma antes de apagar un clock se deberá deshabilitar el modulo.

- El registro **SCGC5** del modulo SIM contiene los bits necesarios para habilitar el clock de la lógica de PORT.



Port X Clock Gate control

This bit controls the clock gate to the port X module

0 Clock disabled
1 Clock enabled

X= A,B,C,D,E

```
MK64F12.h

1  /*! @name SCGC5 - System Clock Gating Control Register 5 */
2  #define SIM_SCGC5_LPTMR_MASK          (0x1U)
3  #define SIM_SCGC5_LPTMR_SHIFT         (0U)
4  ...
5  #define SIM_SCGC5_PORTA_MASK          (0x200U)
6  #define SIM_SCGC5_PORTA_SHIFT         (9U)
7  ...
8  #define SIM_SCGC5_PORTB_MASK          (0x400U)
9  #define SIM_SCGC5_PORTB_SHIFT         (10U)
10 ...
11 #define SIM_SCGC5_PORTC_MASK          (0x800U)
12 #define SIM_SCGC5_PORTC_SHIFT         (11U)
13 ...
```

1.8 - Proramming PORTx_ISFR

Antes de trabajar con cualquier GPIO:

- GATING** Enable clock for used ports
- PORT** Configure MUX & PIN
- GPIO** Configure I/O

MK64F12.h

```

1  #define __I volatile const
2  #define __O volatile
3  #define __IO volatile
4  GPIO Peripheral Access Layer
5  /** GPIO - Register Layout Typedef */
6  typedef struct {
7  __IO uint32_t PDOR; /**< Port Data Output Register, offset: 0x0 */
8  __O uint32_t PSOR; /**< Port Set Output Register, offset: 0x4 */
9  __O uint32_t PCOR; /**< Port Clear Output Register, offset: 0x8 */
10 __O uint32_t PTOR; /**< Port Toggle Output Register, offset: 0xC */
11 __I uint32_t PDIR; /**< Port Data Input Register, offset: 0x10 */
12 __IO uint32_t PDDR; /**< Port Data Direction Register, offset: 0x14 */
13 } GPIO_Type ;

```

Ejemplo de Clock-Gating:

```

1  //Enable clocking for port B
2  SIM->SCGC5 |= SIM_SCGC5PORTB_MASK
3
4  PTB->PCOR = (1<<21)|(1<<22); // Clear pin 21 and 22
5  PTB->PSOR = (1<<21)|(1<<22); // Set pin 21 and 22
6  PTB->PDOR = (PTB->PDOR & ~(1<<pin)) | (data << pin); // Change pin value
7  PTB->PDDR = (1<<21)|(1<<22); // Defino pin 21 y 22 del GPIO B como Salidas
8  PTB->PTOR = (1<<21)|(1<<22); // Toggle pin 21 and 22
9
10 if (PTA->PDIR & (1<<4)) // Test pin 4
11   Do_something(); // if pin 4 == 1

```

1.9 - PORT_type

MK64F12.h

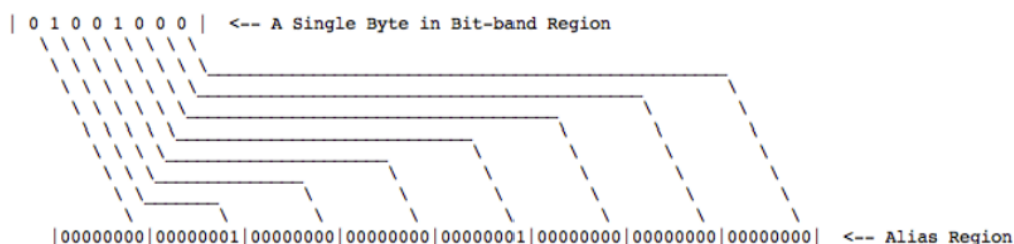
```

1  /** PORT Peripheral Access Layer
2  /** PORT - Register Layout Typedef */
3  typedef struct {
4  __IO uint32_t PCR[32]; /**< Pin Control Register n, array offset: 0x0, array step: 0x4 */
5  __O uint32_t GPCLR; /**< Global Pin Control Low Register, offset: 0x80 */
6  __O uint32_t GPCHR; /**< Global Pin Control High Register, offset: 0x84 */
7  uint8_t RESERVED_0[24];
8  __IO uint32_t ISFR; /**< Interrupt Status Flag Register, offset: 0xA0 */
9  uint8_t RESERVED_1[28];
10 __IO uint32_t DFER; /**< Digital Filter Enable Register, offset: 0xC0 */
11 __IO uint32_t DFCR; /**< Digital Filter Clock Register, offset: 0xC4 */
12 __IO uint32_t DFWR; /**< Digital Filter Width Register, offset: 0xC8 */
13 } PORT_Type;
14

```

2 - Clase 2

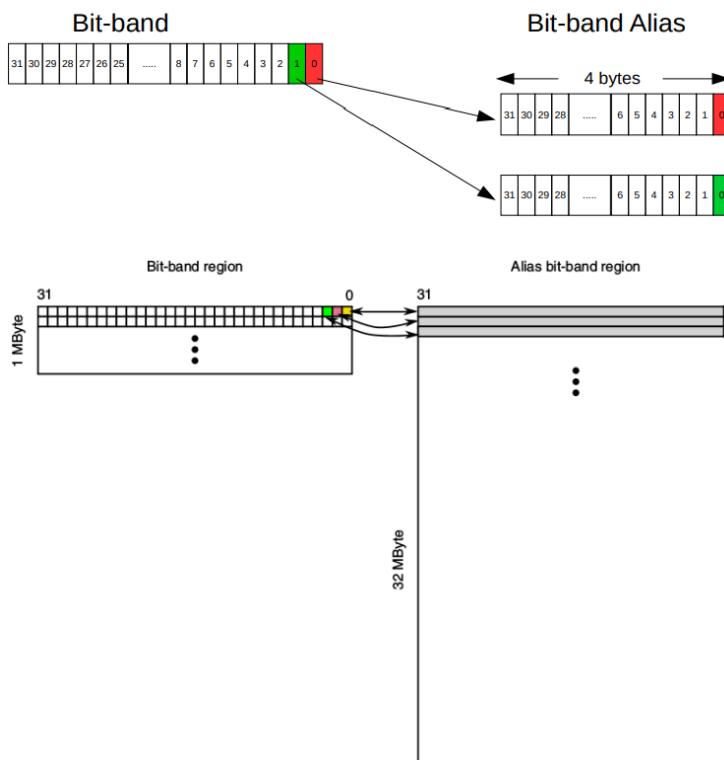
2.1 - Bit Band



Una CPU no puede modificar bits individuales de una posición de memoria (o de un registro). La CPU solo puede modificar bytes o words completos en cada acceso. Si la CPU desea modificar un bit de un registro (o posición de memoria), debe hacer una copia de este último en un registro temporario, modificar dicho bit mediante operaciones lógicas y finalmente escribir el valor modificado al lugar original. Esta forma de modificar bits individuales usando operaciones del tipo Read-Modify-Write (no-atómicas) funciona bien cuando se hace una operación a la vez, pero puede generar problemas cuando dos o más aplicaciones acceden de manera concurrente a ese registro (o posición de memoria). Por ejemplo, ¿qué ocurre si una interrupción se produce entre la lectura y la modificación? El nuevo valor será sobrescrito por la interrupción.

La solución de Bit-Banding: El bit-banding resuelve este conflicto mapeando cada bit individual de una zona de memoria llamada **Bit-band Region** a una palabra completa de 32 bits en otra zona de memoria llamada **Alias Region**. Al escribir una palabra completa en la dirección de la zona alias, el hardware del microcontrolador traduce automáticamente esa escritura a una de lectura-modificación-escritura atómica en un único ciclo de bus sobre el bit correspondiente en la región original.

- **Escritura:** Si escribís un valor en una dirección del alias, solo el bit menos significativo (bit 0) del dato escrito determina el nuevo estado del bit real. Escribir un valor con el bit 0 en 1 (como 0x01 o 0xFF) pone el bit real en 1. Escribir un valor con el bit 0 en 0 (como 0x00 o 0x0E) pone el bit real en 0.
- **Lectura:** Al leer una palabra en la dirección del alias, el hardware devuelve 0x01 si el bit real está en 1, o 0x00 si el bit real está en 0. Los bits del 1 al 31 se retornan siempre como 0.



2.2 - Interrupciones

Una interrupción es un evento que suspende la ejecución normal del programa **forzando** al procesador a realizar otra “tarea” de mayor prioridad que una vez finalizada retorna al programa suspendido.

- Dicho evento es generado por el hardware y se lo conoce como **Interrupt Request (IRQ)**. Este pedido es asíncrono al programa actualmente en ejecución.
- La transferencia se realiza de manera automática.
- La tarea es conocida como Interrupt Service Routine (**ISR**)

El programa en ejecución recibe de manera asíncrona el pedido de interrupción. **Dicha interrupción será aceptada al finalizar la instrucción actual de assembler (no de C)**. La latencia (tiempo de respuesta) del software es el tiempo que tarda desde que ocurre el evento hasta que el mismo es atendido.

2.2.1 - Máscaras

Las interrupciones del procesador pueden ser inhibidas o habilitadas por software. La terminología frecuentemente usada es enmascarar (inhibir) una interrupción. Existen dos tipos de máscaras.

- **Máscara global:** Una llave general que corresponde a un bit del procesador (Global Interrupt Enable).
- **Máscara individual:** Una llave o **enable** para cada periférico del microcontrolador.

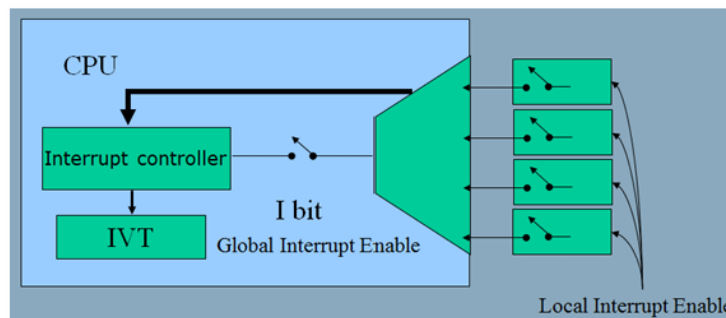
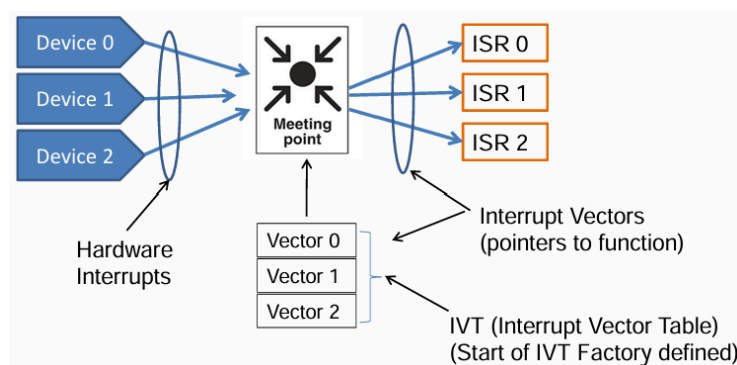


Figura 31: Existen dos tipos de interrupciones: Enmascarables y no-enmascarables por software

2.2.2 - Interrupt Vectors

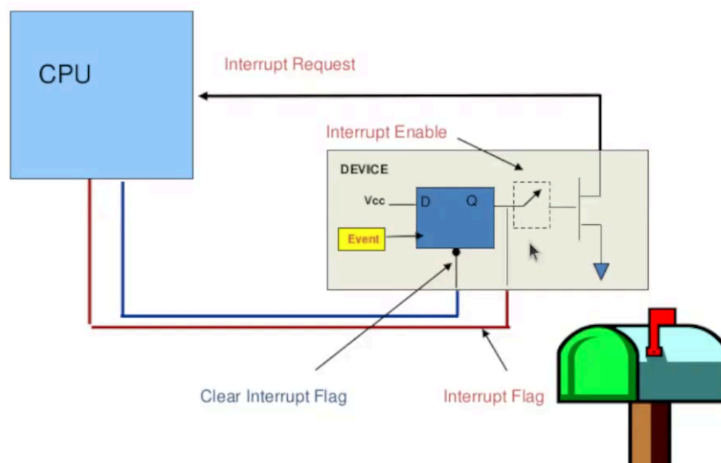
Un vector de interrupción es básicamente un puntero a una función. Es la dirección de memoria donde comienza el código de la rutina de servicio de interrupción (ISR), que es el programa que la CPU debe ejecutar cuando ocurre un evento de hardware específico.

La tabla de vectores de interrupción (IVT) es un arreglo o tabla en memoria donde se almacenan todos estos punteros de manera ordenada. Cuando se activa una interrupción, el hardware del procesador consulta esta tabla de forma automática para saber a qué dirección saltar



2.2.3 - Flags

Un periférico activa la interrupción colocando un 1 en D y por lo tanto, 0. CPU puede **leer estado de flag**, **resetear estado de flag** para terminar con la interrupción



2.2.4 - Secuencia de la Interrupcion

Configuración inicial (**Software**)

1. Se habilita la interrupción particular (Periferico).
2. Se desenmascaran las interrupciones Globales.

Entrada a la rutina de interrupción (**Hardware**)

1. Se genera el pedido de interrupción, encendiéndose el flag correspondiente (xxxIF).
2. Se finaliza la ejecución de la instrucción en curso.
3. Se guarda en el stack el contexto de trabajo (dirección de retorno y demás registros) en forma automática.
4. Se transfiere el control a la ISR, mediante el vector de interrupción.

2.2.5 - Código Bloqueante vs Pooling

Supongamos que un compañero va a pasar a visitarnos dentro de una o dos horas. El momento exacto no lo sabe pero quedamos en que me enviara un mensaje por WhatsApp cuando este arribando a nuestra casa. El problema es que mi celular se quedo sin audio después de finalizada la conversación así que no puedo saber en que momento me enviara el mensaje.

Posibles Soluciones

- Observar la pantalla hasta que aparezca el mensaje (esta solución me impediría o bloquearía la posibilidad de realizar otras tareas)
- Observar la pantalla del celular de vez cuando (esto resolvería el problema anterior permitiéndome intercalar otras tareas)

Código Bloqueante: La CPU interroga constantemente al periférico para ver si hay algún evento que atender.

Pooling: La CPU interroga al periférico intercalando ahora otras tareas

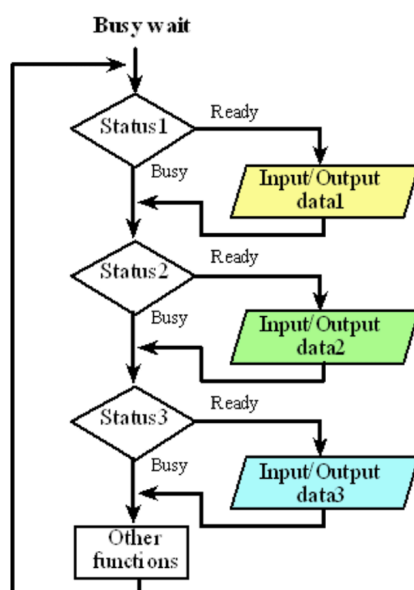


Figura 34: Diagrama de Pooling

Aunque resuelve el bloqueo total al permitir realizar más de una función, sigue siendo una estrategia ineficiente debido a que la CPU realiza consultas a una frecuencia extremadamente alta (en microsegundos) para eventos externos que ocurren muy lentamente (como un usuario pulsando un botón en el orden de los milisegundos).

2.2.6 - Periodic Interrupt / Hardware Polling

Gran parte de los eventos generados por el hardware son lentos en relación a la velocidad del procesador para este caso la mejor estrategia es utilizar una interrupción periódica (PISR).

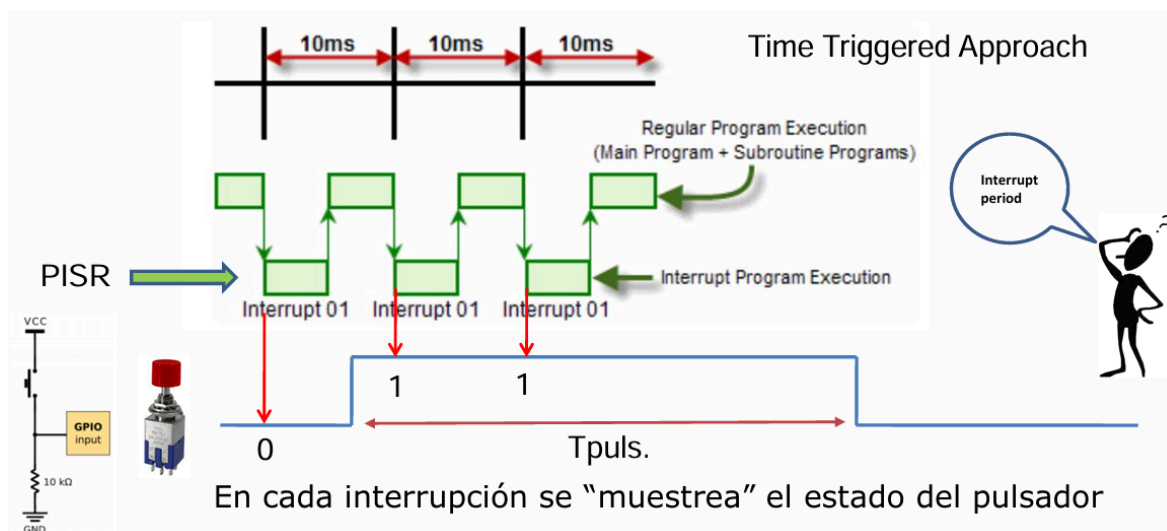
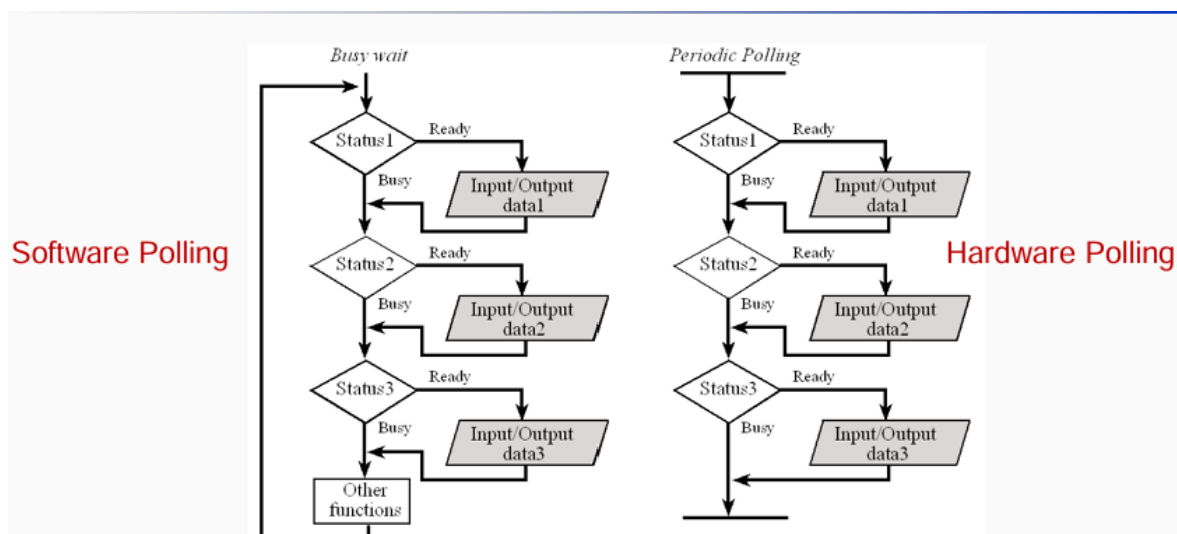


Figura 35: Para interrupciones como un boton conviene tener un timer aparte de mas tiempo

En lugar de consultar continuamente en el lazo principal, se configura un temporizador físico de hardware (como SysTick o un canal del módulo PIT) para que genere una interrupción a intervalos de tiempo fijos y controlados (por ejemplo, cada 10 milisegundos).

Al dispararse la interrupción periódica, la CPU suspende momentáneamente el programa principal, ejecuta una rutina de servicio rápida (ISR) donde realiza el chequeo del estado del periférico (hace el polling sincronizado por hardware), y regresa de inmediato al flujo principal.

Si el periférico no requiere atención, la rutina finaliza inmediatamente sin perturbar el procesamiento del lazo principal, reduciendo drásticamente la subutilización del chip.



2.2.7 - Interrupciones Dedicadas

Cuando el evento es de corta duración en relación al tiempo de la CPU una interrupción periódica no es posible. En estos casos deberemos utilizar una interrupción dedicada a ese evento.

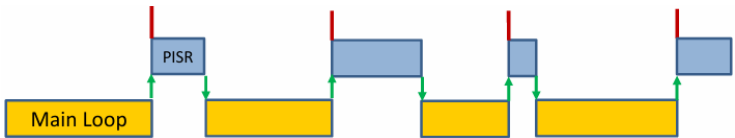


Figura 37: Periódicas

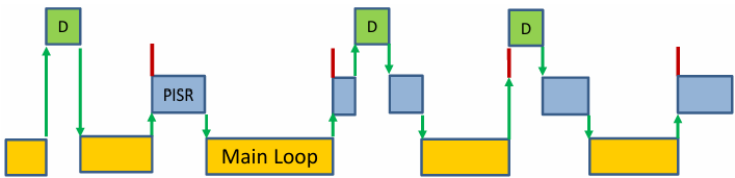
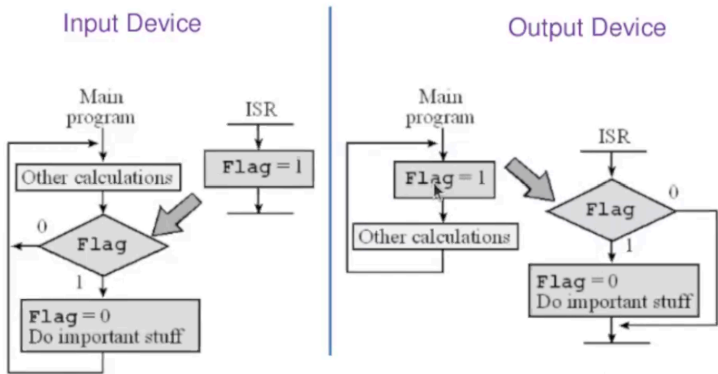


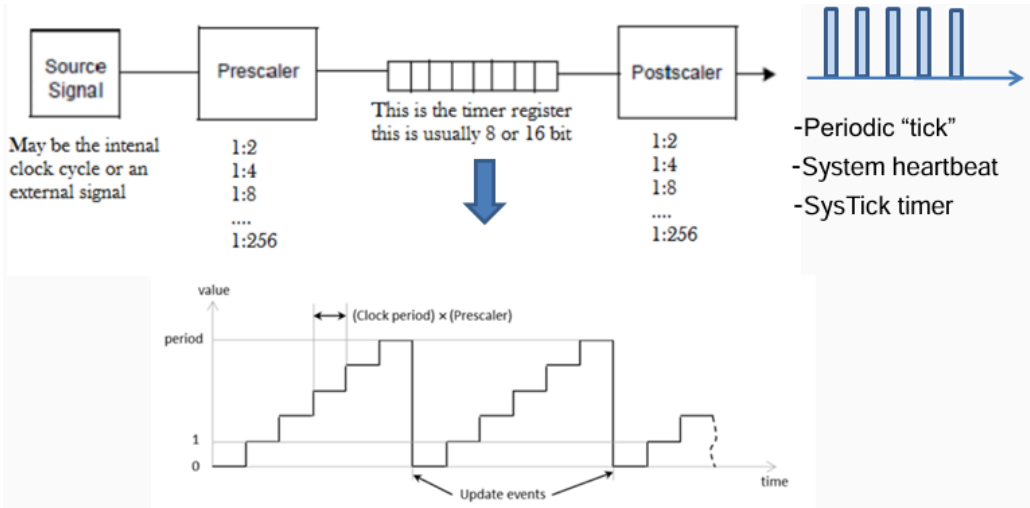
Figura 38: Periódicas + Dedicadas

Comunicación entre el programa principal y la ISR con periféricos de entrada y salida.



2.2.8 - Fuentes de interrupción periódica

Todos los microcontroladores tienen un generador periódico de interrupciones. En general se los puede configurar por programa para modificar el periodo de interrupción



2.3 - Interrupciones del K64

2.3.1 - NVIC / Nested Vector Interrupt Controller

La arquitectura Cortex M4 tiene un control avanzado de interrupciones y excepciones. El NVIC recibe excepciones del sistema así como interrupciones externas

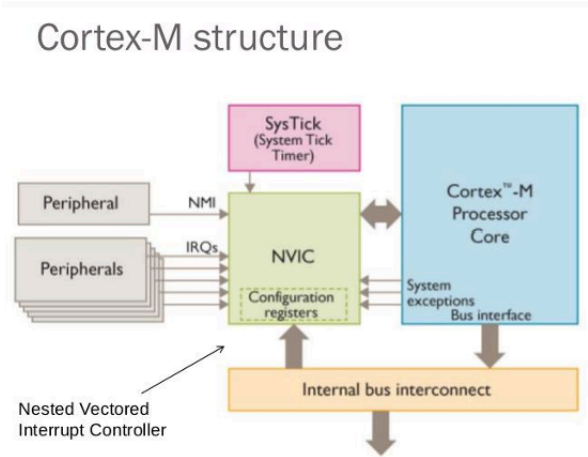


Figura 41: CMSIS: Cortex Microcontroller Software Interface Standard

Es el controlador de interrupciones anidadas y vectorizadas integrado directamente dentro de la CPU Cortex-M4. A diferencia de otros sistemas de hardware donde el control de interrupciones es externo, el NVIC está fuertemente acoplado con la CPU. Esto permite tiempos de respuesta sumamente veloces y una integración optimizada con los modos de bajo consumo del microcontrolador.

Table 7.1 List of System Exceptions			
Exception Number	Exception Type	Priority	Description
1	Reset	−3 (Highest)	Reset
2	NMI	−2	Nonmaskable interrupt (external NMI input)
3	Hard fault	−1	All fault conditions if the corresponding fault handler is not enabled
4	MemManage fault	Programmable	Memory management fault; Memory Protection Unit (MPU) violation or access to illegal locations
5	Bus fault	Programmable	Bus error; occurs when Advanced High-Performance Bus (AHB) interface receives an error response from a bus slave (also called <i>prefetch abort</i> if it is an instruction fetch or <i>data abort</i> if it is a data access)
6	Usage fault	Programmable	Exceptions resulting from program error or trying to access coprocessor (the Cortex-M3 does not support a coprocessor)
7–10	Reserved	NA	—
11	SVC	Programmable	Supervisor Call
12	Debug monitor	Programmable	Debug monitor (breakpoints, watchpoints, or external debug requests)
13	Reserved	NA	—
14	PendSV	Programmable	Pendable Service Call
15	SYSTICK	Programmable	System Tick Timer

Figura 42: ARM CORE System Exceptions

Table 7.2 List of External Interrupts		
Exception Number	Exception Type	Priority
16	External Interrupt #0	Programmable
17	External Interrupt #1	Programmable
...	...	...
255	External Interrupt #239	Programmable

Figura 43: External System Exceptions

Para controlar y supervisar las interrupciones de periféricos del K64, el NVIC utiliza un conjunto estructurado de registros agrupados en la estructura CMSIS NVIC_Type:

- **ISER (Interrupt Set-Enable Registers):** Registros de 32 bits utilizados para habilitar individualmente cada línea de interrupción. El bit asignado en ISER a un periférico debe ponerse en 1 para que el procesador comience a escuchar ese evento.

- **ICER (Interrupt Clear-Enable Registers):** Registros para deshabilitar interrupciones específicas de forma individual.
- **ISPR (Interrupt Set-Pending Registers):** Registros que colocan una interrupción en estado pendiente. Esto permite disparar una interrupción por software de manera controlada para pruebas o programación estructurada.
- **ICPR (Interrupt Clear-Pending Registers):** Registros que limpian el estado pendiente de las interrupciones individuales.
- **IABR (Interrupt Active Bit Registers):** Registros que informan si una interrupción está activa en ese momento (ejecutando su respectivo Handler) o si ha sido pausada para atender a una de mayor prioridad.
- **IPR o IP (Interrupt Priority Registers):** Registros encargados de fijar la prioridad de cada interrupción. Aunque el procesador Cortex-M4 admite prioridades de 8 bits completos, el chip K64 solo implementa los 4 bits más significativos, permitiendo elegir entre 16 niveles de prioridad (**donde 0 es la máxima prioridad y 15 es la mínima**)

```
core_cm4.h
1  typedef struct
2  {
3      __IOM uint32_t ISER[8U];          /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register */
4          uint32_t RESERVED0[24U];
5      __IOM uint32_t ICER[8U];          /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register */
6          uint32_t RESERVED1[24U];
7      __IOM uint32_t ISPR[8U];          /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register */
8          uint32_t RESERVED2[24U];
9      __IOM uint32_t ICPR[8U];          /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register */
10         uint32_t RESERVED3[24U];
11      __IOM uint32_t IABR[8U];          /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register */
12         uint32_t RESERVED4[56U];
13      __IOM uint8_t IP[240U];           /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit
wide) */
14         uint32_t RESERVED5[644U];
15      __IOM uint32_t STIR;              /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register
*/
16 } NVIC_Type;
```

2.3.2 - Weak

El startup puede declarar handlers por defecto.

```
1 void PORTA_IRQHandler(void) __attribute__((weak));
```

“Esta función existe como implementación por defecto, pero si el usuario proporciona otra con el mismo nombre, usá la del usuario.”

```
1 void PORTA_IRQHandler(void)
2 {
3     // MI código
4 }
```

2.3.3 - CMSIS - Functions

Es una forma estandarizada de acceder al Cortex-M desde C para no necesitar calcular manualmente los registros todo el tiempo.

En vez de hacer:

```
1 NVIC->ISER[1] = (1 << 26);
```

Podemos utilizar

```
1 NVIC_EnableIRQ(PORTA_IRQn);
```

core_cm4.h

```

1  /* ##### NVIC functions ##### */
2  /**
3   \ingroup CMSIS_Core_FunctionInterface
4   \defgroup CMSIS_Core_NVICFunctions NVIC Functions
5   \brief Functions that manage interrupts and exceptions via the NVIC.
6   @{
7   */
8   #define NVIC_SetPriorityGrouping    __NVIC_SetPriorityGrouping
9   #define NVIC_GetPriorityGrouping    __NVIC_GetPriorityGrouping
10  #define NVIC_EnableIRQ              __NVIC_EnableIRQ
11  #define NVIC_GetEnableIRQ           __NVIC_GetEnableIRQ
12  #define NVIC_DisableIRQ             __NVIC_DisableIRQ
13  #define NVIC_GetPendingIRQ          __NVIC_GetPendingIRQ
14  #define NVIC_SetPendingIRQ          __NVIC_SetPendingIRQ
15  #define NVIC_ClearPendingIRQ        __NVIC_ClearPendingIRQ
16  #define NVIC_GetActive              __NVIC_GetActive
17  #define NVIC_SetPriority            __NVIC_SetPriority
18  #define NVIC_GetPriority            __NVIC_GetPriority
19  #define NVIC_SystemReset           __NVIC_SystemReset

```

core_cm4.h

```

1  /**
2   \brief Set Interrupt Priority
3   \details Sets the priority of a device specific interrupt or a processor exception.
4           The interrupt number can be positive to specify a device specific interrupt,
5           or negative to specify a processor exception.
6   \param [in] IRQn Interrupt number.
7   \param [in] priority Priority to set.
8   \note The priority cannot be set for every processor exception.
9   */
10 __STATIC_INLINE void __NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)
11 {
12     if ((int32_t)IRQn >= 0)
13     {
14         NVIC->IP[((uint32_t)IRQn)] = (uint8_t)((priority << (8U - __NVIC_PRIO_BITS)) &
15         (uint32_t)0xFFUL);
16     }
17     else
18     {
19         SCB->SHP[(((uint32_t)IRQn) & 0xFUL)-4UL] = (uint8_t)((priority << (8U - __NVIC_PRIO_BITS)) &
20         (uint32_t)0xFFUL);
21     }
22 }

```

2.3.4 - Ejemplo de Implementacion de Interrupcion

App.c

```

1  __ISR__ SW3_Handler (void){
2  PORT_ClearInterruptFlag (PA, 4);
3  gpioToggle(PIN_LED_BLUE);
4  }

1  /* Función que se llama 1 vez, al comienzo del programa */
2  void App_Init (void){
3  hw_DisableInterrupts(); //para que no me interrumpan la inicializacion
4
5  //Habilito los clocks de solo los puertos que voy a usar
6  SIM->SCGC5 |= SIM_SCGC5_PORTB_MASK;
7  SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK;
8  SIM->SCGC5 |= SIM_SCGC5_PORTE_MASK;
9
10 //LED VERDE
11 PORTE->PCR[26]=0x0; //Clear all bits
12 PORTE->PCR[26]|=PORT_PCR_MUX(1); //Set MUX to GPIO that is ALT1
13 PORTE->PCR[26]|=PORT_PCR_DSE(1); //Drive strength enable
14
15 // Global Pin Control High Registe Which Pins. Me ahorro:
16 // PORTB->PCR[21] = ...
17 // PORTB->PCR[22] = ...
18 uint32_t temp = PORT_GPCHR_GPWE((1<<(21-16))|(1<<(22-16)));
19 temp|=PORT_GPCHR_GPWD(PORT_PCR_MUX(1)); //Set MUX to GPIO
20
21 // Now set pins output properties
22 temp|=PORT_GPCHR_GPWD(PORT_PCR_DSE(1)); //Drive strength enable
23 PORTB->GPCHR=temp; //Set all at once
24
25 //SW3 - Pin: PTA4
26 PORTA->PCR[4]=0x0; //Clear
27 PORTA->PCR[4]|=PORT_PCR_MUX(1); //Set MUX to GPIO
28 PORTA->PCR[4]|=PORT_PCR_PE(1); //Pull UP/Down Enable
29 PORTA->PCR[4]|=PORT_PCR_PS(1); //Pull UP
30 // Enable interrupt on this pin (PTA4)
31 PORTA->PCR[4]|=PORT_PCR_IRQC(PORT_eInterruptRising); //Enable Rising edge interrupts
32
33 // Nos aseguramos de que todo arranque apagado
34 gpioWrite(PIN_LED_RED, !LED_ACTIVE);
35 gpioWrite(PIN_LED_BLUE, !LED_ACTIVE);
36
37 gpioWrite(PIN_LED_GREEN, LED_ACTIVE);
38 SysTick_Init();
39 hw_EnableInterrupts();
40 }

```

SysTick.c

```

1 // Puntero a función (callback) para ejecutar la rutina del usuario
2 static void (*p_callback)(void) = 0;

1 /* Inicializa el SysTick para generar ticks periódicos y guarda el callback */
2 void SysTick_Init(void (*func)(void))
3 {
4     // 1. Guardar la función que se llamará en cada interrupción
5     p_callback = func;
6     // 2. Deshabilitar el temporizador antes de configurar
7     SysTick->CTRL = 0x00;
8     // 3. Cargar el valor de cuenta para 125 ms @ 100 MHz (12,500,000 ciclos - 1)
9     SysTick->LOAD = 12500000L - 1;
10    // 4. Limpiar el valor actual del contador
11    SysTick->VAL = 0x00;
12    // 5. Configurar fuente de reloj del núcleo, habilitar interrupción y activar SysTick
13    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
14                   SysTick_CTRL_TICKINT_Msk |
15                   SysTick_CTRL_ENABLE_Msk;
16 }

1 /* ISR nativa del procesador para SysTick */
2 void SysTick_Handler(void)
3 {
4     // Ejecutar el callback del usuario si fue registrado en Init
5     if (p_callback != 0)
6     {
7         p_callback();
8     }
9 }

```

gpio.h

```

1 // Ports
2 enum { PA, PB, PC, PD, PE };
3
4 // Convert port and number into pin ID
5 // Ex: PTB5 -> PORTNUM2PIN(PB,5) -> 0x25
6 //      PTC22 -> PORTNUM2PIN(PC,22) -> 0x56
7 #define PORTNUM2PIN(p,n) (((p)<<5) + (n))
8 #define PIN2PORT(p) (((p)>>5) & 0x07)
9 #define PIN2NUM(p) ((p) & 0x1F)

```

gpio.c

```

1 //Escribe un valor HIGH o LOW en un pin digital
2 void gpioWrite (pin_t pin, bool value){
3     uint8_t portId = PIN2PORT(pin);
4     uint8_t pinNum = PIN2NUM(pin);
5
6     if (value) {gpio_ptr[portId]->PSOR = (1 << pinNum);} // Port Set Output Register
7     else {gpio_ptr[portId]->PCOR = (1 << pinNum);} // Port Clear Output Register
8 }

1 // Invierte el valor de un pin digital (HIGH<->LOW)
2 void gpioToggle (pin_t pin){
3     uint8_t portId = PIN2PORT(pin);
4     uint8_t pinNum = PIN2NUM(pin);
5
6     gpio_ptr[portId]->PTOR = (1 << pinNum); // Port Toggle Output Register
7 }

1 //Lee el valor de un pin digital especificado
2 bool gpioRead (pin_t pin)
3 {
4     uint8_t portId = PIN2PORT(pin);
5     uint8_t pinNum = PIN2NUM(pin);
6
7     return (gpio_ptr[portId]->PDIR & (1 << pinNum)) != 0; // Port Data Input Register
8 }

1 //Inicializa como interrupcion al pin, con el modo y hacia la func handler que le mandes. - IRQ
2 bool gpioIRQ(pin_t pin, uint8_t irqMode, pinIrqFun_t irqFun) {
3     uint8_t portId = PIN2PORT(pin);
4     uint8_t pinNum = PIN2NUM(pin);
5
6     if (portId > PE || irqMode >= GPIO_IRQ_CANT_MODES) {return false;}
7
8     irq_callbacks[portId][pinNum] = irqFun;
9
10    static const uint8_t irqc_map[] = {0x0, 0x9, 0xA, 0xB};
11
12    uint32_t irqc_value = irqc_map[irqMode]; // Obtenemos el valor directo sin switch
13
14    static PORT_Type * const puertos_base[] = {PORTA, PORTB, PORTC, PORTD, PORTE};
15
16    puertos_base[portId]->PCR[pinNum] = (puertos_base[portId]->PCR[pinNum] & ~PORT_PCR_IRQC_MASK) |
17                                     PORT_PCR_IRQC(irqc_value);
18
19    //Se para en la posicion base del primer puerto y se mueve port id veces hasta tu puerto
20    NVIC_EnableIRQ(PORTA_IRQn + portId);
21
22    return true;
23 }

```

3 - Clase 3

3.1 - Prioridad de Interrupciones

1. Tail-Chaining

Al terminar de ejecutar una rutina de servicio de interrupción (ISR), si existe otra interrupción pendiente con la prioridad suficiente para ser atendida, el procesador evita realizar la secuencia completa de desapilado de registros (pop) y posterior apilado (push)

En lugar de desperdiciar ciclos de reloj restaurando el contexto para luego volver a guardarlo, la CPU salta directamente a la ejecución del nuevo manejador pendiente

Gracias a esto, la transición entre interrupciones consecutivas demora únicamente 6 ciclos de reloj en condiciones de memoria sin estados de espera, en comparación con los más de 20 ciclos que requeriría hacer el desapilado y apilado completo

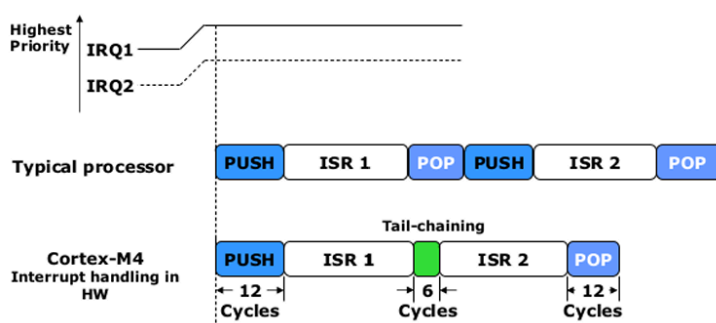


Figura 44: Interrupt Latency - Tail Chaining

2. Pre-Emption

Para que ocurra el desalojo, la nueva interrupción entrante debe tener una prioridad de grupo estrictamente mayor (es decir, un número de prioridad menor en el registro de configuración) que la interrupción que se está ejecutando

Si la nueva interrupción tiene una prioridad de grupo igual o menor, no se produce el desalojo; la nueva interrupción simplemente permanece en estado pendiente hasta que el procesador finalice la ejecución del manejador actual

Cuando se produce el desalojo, las interrupciones se anidan, y el procesador guarda automáticamente el contexto en la pila para retomar la tarea suspendida de forma segura más tarde

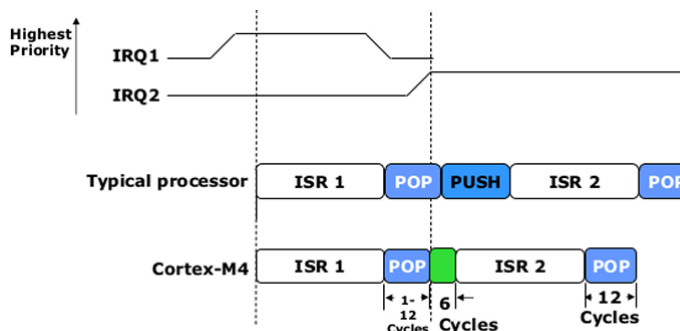


Figura 45: Interrupt Latency - Pre-Emption

3. Late Arrival

Si una interrupción de alta prioridad se activa durante la fase en la que el procesador está realizando el apilado automático de registros para una interrupción previa de menor prioridad, el hardware no detiene ni descarta el proceso de apilado

Como el estado que debe guardarse en la pila es idéntico sin importar cuál de las dos interrupciones se ejecute primero, el apilado de registros continúa sin verse afectado

Sin embargo, en paralelo a esa acción, el procesador cambia la búsqueda del vector en la tabla y salta directamente a ejecutar primero la rutina de la interrupción de mayor prioridad (la que llegó tarde)

El hardware permite aceptar esta interrupción de llegada tardía hasta antes de que la primera instrucción del manejador original de menor prioridad entre en la etapa de ejecución de la CPU

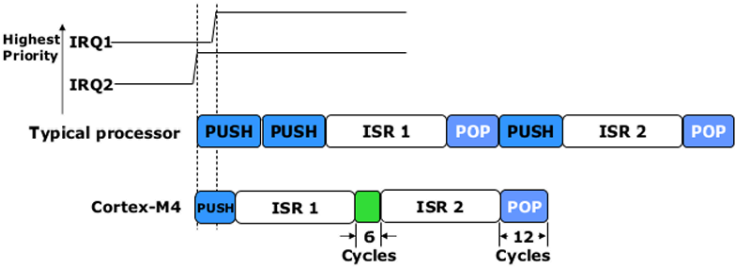


Figura 46: Interrupt Latency - Late Arrival

4 - Clase 4

4.1 - Arquitectura de Firmware

Al desarrollar sistemas embebidos complejos, se manejan múltiples periféricos y tareas concurrentes. Para evitar que el firmware se vuelva inmanejable y difícil de depurar, se emplea un **diseño modular estructurado en capas de abstracción bien definidas**.

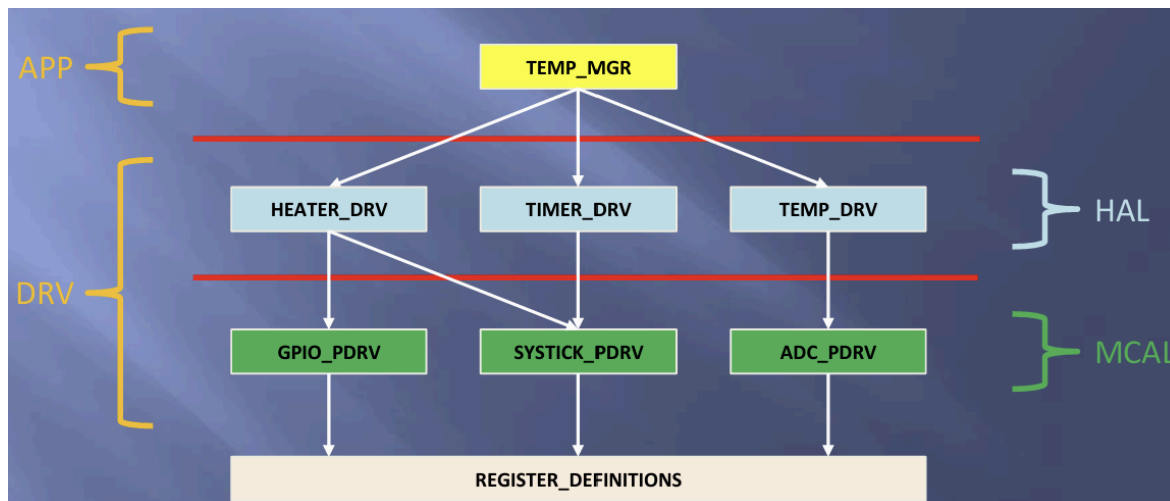


Figura 47: Ejemplo de Arquitectura de Firmware: Controlador de temperatura

4.1.1 - Capas e Interfaces

La arquitectura organiza los distintos módulos en una jerarquía clara:

- **Capa de Aplicación (APP):** Es la capa de mayor jerarquía. Contiene la lógica del negocio o del funcionamiento del equipo. Sigue las reglas de la programación tradicional y es independiente del tiempo real. Nunca debe interactuar con los periféricos ni leer o escribir sus registros de forma directa; se comunica únicamente a través de los servicios de los drivers.
- **Capa de Drivers (DRV):** Son las librerías de hardware encargadas de configurar y controlar los periféricos físicos. Resuelven las restricciones temporales del sistema y agregan versatilidad. Se subdividen en:
 - ▶ **Peripheral Drivers (MCAL):** Capa de Abstracción del Microcontrolador. Configura directamente los registros de los periféricos internos del chip (como PORT, ADC, PIT o FTM). Al migrar de microcontrolador, solo esta capa debe modificarse. **Capa mas baja de escritura, si cambio el procesador, solo tengo que modificar esta capa.**
 - ▶ **Hardware Drivers (HAL):** Capa de Abstracción de Hardware. Se encarga de controlar los elementos externos que están conectados al microcontrolador (como un sensor de temperatura externo, displays o un driver de motor). Al cambiar un componente externo, solo se altera esta capa. **Capa mas baja de escritura, si cambio el procesador, solo tengo que modificar esta capa.**
 - ▶ **Complex Drivers:** Módulos integrados que unifican el control de registros y hardware externo debido a limitaciones de tiempo extremadamente críticas o simplicidad del sistema.

4.2 - Estructura de Código

La estructura del código de los drivers posee 4 partes:

- Inicialización (Init)
- Servicios (Srv)
- Interrupción periódica (PISR)
- Interrupción dedicada (ISR)

4.2.1 - DRV_Init()

- Configura inicial del periférico y variables propias.
- Se llama una sola vez, al inicio del programa.
- Puede no estar presente (si no es necesario por el DRV).

4.2.2 - DRV_SrvN()

Nunca acceder de forma directa a los registros. Siempre a través de estos servicios.

- Interfaz con la capa inmediata superior.
- Permite comenzar tareas, leer eventos y estado de los periféricos o hardware, sin acceder directamente a los registros.
- Pueden haber tantas funciones como servicios se desea brindar.
- Pueden ser llamadas con cualquier frecuencia y periodicidad.
- Puede agregar (emular) funcionalidades que no tenga el periférico.

4.2.3 - DRV_PISR()

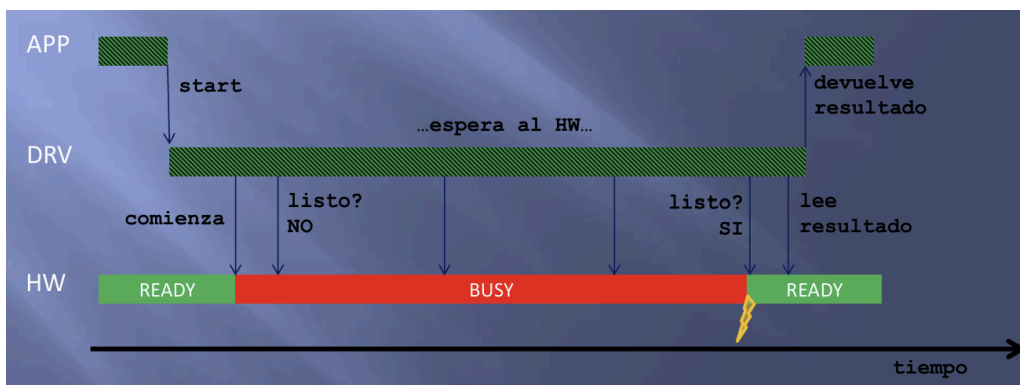
- Función que debe ser llamada periódicamente, a frecuencia definida por el driver.
- Realiza las tareas periódicas del DRV.
- Para **eventos lentos** (si es posible, es preferible un hardware polling a una interrupción dedicada).
- Puede no estar presente (si no es necesario por el DRV).

4.2.4 - DRV_ISR()

- Función que es llamada por el periférico al ocurrir un evento.
- Necesario para eventos rápidos.
- Puede no estar presente (si no es necesario por el DRV).

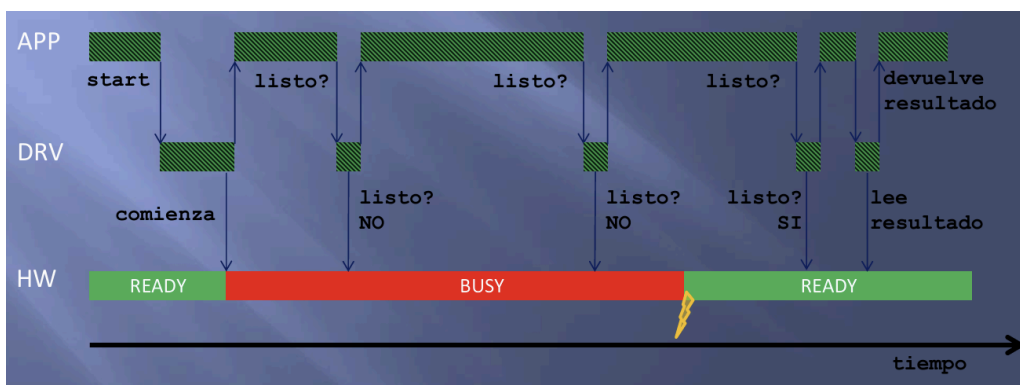
4.3 - Analisis temporal

4.3.1 -Codigo Bloqueante



El DRV bloquea el uso de la CPU hasta que la tarea finalice / evento ocurra, y le devuelve a la APP el resultado.

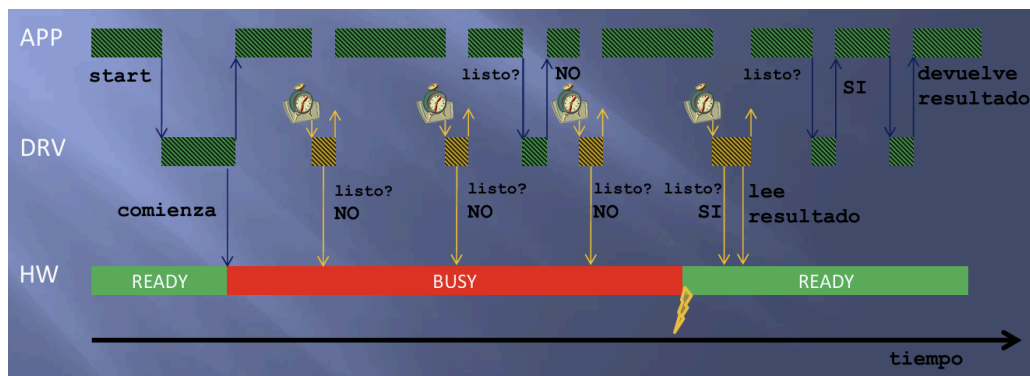
4.3.2 - Polling



- Secuencia de 3 servicios del DRV: comenzar, consultar estado, leer resultado.
- El DRV funciona «de pasamanos», solo enmascara los registros del HW

4.3.3 - PISR

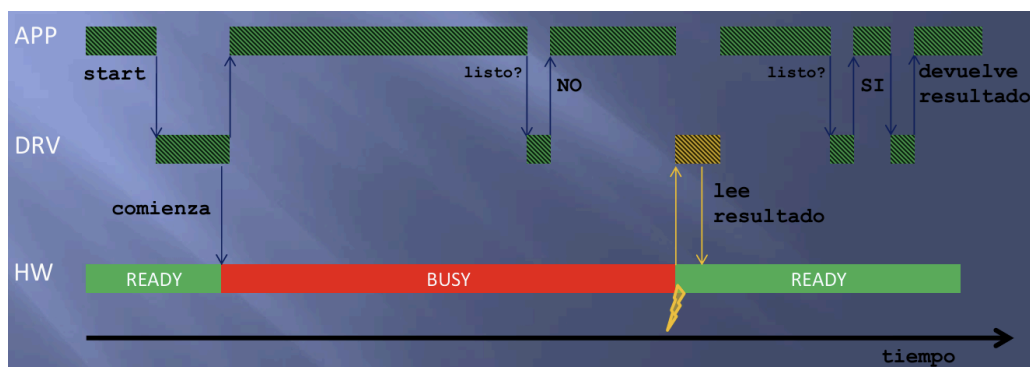
Interrupción de timer periódica (rayado amarillo y negro). La app queda suspendida por un tiempo muy ínfimo en comparación al polling al ser la interrupción la que interroga al hardware.



- Secuencia de 3 servicios del DRV: comenzar, consultar estado, leer resultado.
- No hay conflicto si solo ocurre 1 evento entre un período del PISR.
- El evento es capturado por el DRV y almacenado hasta que la APP lo solicite.

4.3.4 - ISR (Interrupcion dedicada)

App llama al driver para que empiece la interrupcion. La app llama al driver para ver si esta listo o no (a traves de un `getStatus()` por ejemplo). El hardware genera una interrupcion que eleva la señal al driver para levantar un flag para que la app lo lea con un `getStatus()` y devuelva resultado con un `getResult()`.



- El HW notifica al DRV mediante interrupción propia del evento.
- No hay conflicto si solo ocurre 1 evento en el tiempo de ejecución de la ISR.

4.3.5 - ISR + callback

Cada vez que quiera llamar un callback, `DRV_ISR()` debe llamar a `DRV_SrvN()` y no directamente al callback. **Never ever launch a callback from inside a ISR or PISR.**

